Universidade Federal do ABC

Neural Controller for Quadcopter Drones Developed via Genetic Algorithms

Authors:	*RA:*
Ivan Zanutto Bastos	11007116
Matheus Pacheco de Oliveira	11201722352

Advisor: Hugo Puertas de Araújo

April/2025

Abstract

The rapid technological advancements have driven the adoption of autonomous drones across various fields, notably in precision agriculture, logistics, and disaster management, where these devices provide increased efficiency and scalability compared to human-operated systems. In this context, artificial neural networks emerge as a promising solution due to their ability to adapt to dynamic and nonlinear environments. Simultaneously, genetic algorithms (GAs) have proven effective in optimizing neural networks by efficiently exploring solution spaces to adjust weights and parameters.

This study proposes the development of a neural controller for quadcopter drones using GAs, focusing on autonomous navigation in simulated environments. The controller is trained to navigate the drone to designated locations, assuming continuous flight without addressing takeoff or landing. To achieve this, a virtual environment was modeled using the Gymnasium platform, a neural network was implemented for drone control, and genetic algorithms were applied to optimize the controller.

1 Introduction

In recent years, there has been a significant growth in research related to the development and application of drones in various areas, with a focus on the use of autonomous drones. These devices have proven to be particularly promising as they offer a substantial reduction in operational costs and greater scalability compared to operations that require human intervention. Among the main areas of application, precision agriculture stands out, where drones are used for crop monitoring and input spraying; delivery services, which promise greater logistical efficiency by reaching hard-to-access areas; and disaster management, where the speed and accuracy of drones are essential for mapping affected areas, assisting search and rescue operations, and assessing damage. (REJEB et al., 2022; BENARBIA; KYAMAKYA, 2022; DAUD et al., 2022).

In this context, artificial neural networks (ANNs) emerge as a promising solution due to their capacity to learn and generalize complex behaviors from training data, without the need for explicit system models. These networks, inspired by the functioning of the human brain, are particularly effective in dynamic and non-linear environments, where operating variables and conditions can change rapidly. These characteristics make neural networks a powerful tool for developing flight controllers that need to continuously adapt to environmental variables, such as wind or unexpected obstacles. (HAQUE; CHOWDHURY, 2023).

In parallel, the use of Genetic Algorithms (GAs) has proven to be a promising approach for training neural networks. Inspired by the processes of natural selection, GAs are powerful tools for the optimization of parameters in complex systems, such as those involved in adjusting the weights of a neural network. They allow for the efficient exploration of the solution space, finding network configurations that maximize drone performance in different scenarios. (KATOCH; CHAUHAN; KUMAR, 2021; GEN, 1997).

Therefore, this work proposes the combination of neural networks and genetic algorithms as a robust and adaptable solution for the development of an autonomous drone flight controller, with the potential to overcome challenges present in critical applications.

2 Objectives

2.1 General Objective

Develop a neural controller for quadcopter drones using genetic algorithms, capable of autonomous navigation in a virtual environment, moving to a designated target. The controller assumes continuous flight; takeoff and landing are out of scope.

2.2 Specific Objectives

1. **Virtual Environment:** Design and implement a Gymnasium-based environment to reproduce and visualize drone behavior.
2. **Neural Network:** Design and implement an artificial neural network for drone control in simulation.
3. **Genetic Algorithms:** Optimize the controller by tuning parameters and, when applicable, topology.
4. **Simulation and Evaluation:** Assess navigation and obstacle avoidance performance in simulation.

3 Proposed Approach

The project will commence with the modeling of the simulated environment using Gymnasium, an open-source Python library. This environment will be 3-dimensional and configured to include a variety of obstacles and destinations for the drone. It will also define the model quality parameters necessary to calculate our fitness function, thus offering a robust test scenario for the controller.

The neural network will initially be designed with a simple architecture, tailored to handle the real-time control of the drone. The network structure will include inputs corresponding to the drone's sensors (such as distances to obstacles and relative position to the destination) and outputs that will map to the control commands, such as propeller activation and speed.

Genetic Algorithms will be employed to train the neural network, starting with an initial population of networks with random parameters. The performance of each individual will be evaluated based on the drone's ability to reach the destination, or how

close it gets, without colliding with obstacles. The fittest individuals will be selected for reproduction, applying genetic operators such as crossover and mutation to generate the next generation of neural networks. This process will be repeated until the controller achieves a desirable performance or until a maximum number of generations is reached.

Finally, the performance of the trained controller will be evaluated in a series of simulated tests. The drone's success in various environment configurations will be verified.

4 Methodology

4.1 Simulated Environment

The environment was built on top of the *gym-pybullet-drones* project (PANERATI et al., 2021), which uses the *PyBullet* library (COUMANS; BAI, 2016–2019) for physics simulation and the *Crazyflie 2.0* (FÖRSTER, 2015) as the drone model. The main properties of the environment and the changes made to adapt it to the requirements of this work are detailed below.

4.1.1 Observation Space

The neural network observation space was defined as: Relative position to the target (via GPS), velocity (via GPS), acceleration (via accelerometer), orientation quaternion (via gyroscope), depth map (via LIDAR), and altitude (via distance sensor).

4.1.2 Action Space

The action space consists of 4 values ranging from -1 to 1 that control the rotation of each motor individually according to:

$$\text{rpm}(x) = h(1 + 0.05x)$$

where h is the rotation required to keep the drone hovering in place.

4.1.3 Non-conservative Forces

PyBullet has a property called *damping*, which acts as a global friction term for all objects in the simulation. It works as follows:

$$a_d = -d_v v, \text{ and } \alpha_d = -d_\omega \omega$$

where a_d is the linear acceleration caused by the *damping*, d_v is the linear *damping*, and v is the linear velocity. The angular terms are defined analogously.
By default, $d_v = d_\omega = 0.04$. However, the base project *gym-pybullet-drones* includes a drag implementation based on experimental data, which was used instead of the linear *damping*. The angular *damping* was not modified.

4.1.4 Termination and Truncation Criteria

The episode is considered successful if the distance from the drone to the goal reaches the threshold of 0.5 meters.
The episode is truncated if the drone's tilt exceeds the threshold of 0.4 radians or if the distance to the goal exceeds $\sqrt{2}$ times the initial distance.

4.1.5 Fitness Function

Initially, only the distance to the goal was used as an attribute of the *fitness* function, given by:

$$F_d(d) = \begin{cases} 1 - (d/D)^2, & \text{if } d \leq D \\ 0, & \text{otherwise} \end{cases}$$

where d is the final distance and D is the initial distance.

However, to ensure that the drone actually stops at the goal and to encourage trajectory efficiency, the final velocity and total duration were added as attributes. Thus, the final function used is:

$$F(d, v, t) = \begin{cases} 50 \times F_d(d), & \text{if } d \geq \frac{\sqrt{2}}{2} \\ 50 \times F_d(d) + 30 \times F_v(v), & \text{if } d < \frac{\sqrt{2}}{2} \text{ and } v \geq 1 \\ 50 \times F_d(d) + 30 \times F_v(v) + 20 \times F_t(t), & \text{otherwise} \end{cases}$$

where FR_v and F_t are given by:

$$F_v(v) = 1 - (v/V)^2$$

where v is the final velocity and V is the maximum velocity.

$$F_t(t) = 1 - \left(\frac{t - T_{min}}{T_{max} - T_{min}}\right)^2$$

where t is the final time, $T_{max} = \frac{\text{initial distance}}{\text{minimum velocity}}$ and $T_{min} = \frac{\text{initial distance}}{\text{maximum velocity}}$.
The minimum velocity is defined as 1 m/s and the maximum velocity as 20 m/s.

This definition makes the *fitness* work in tiers: velocity only becomes relevant when the distance is smaller than $\frac{\sqrt{2}}{2}$, and time only becomes relevant when the velocity is lower than 1. The values were chosen based on the success threshold (0.5). The idea behind this approach is to impose a priority order on the attributes, which prevents competition between them and speeds up convergence.

Furthermore, the function now ranges from 0 to 100, instead of 0 to 1, to make visualization easier: 50 points are related to distance, 30 to velocity, and 20 to time. These values were chosen according to the importance of each attribute with respect to the desired outcome.

4.2 Genetic Algorithms with DEAP

We first experimented with PyGAD (*Python Genetic Algorithm*) (GAD, 2021). We then adopted DEAP (*Distributed Evolutionary Algorithms in Python*) (FORTIN et al., 2012) for greater flexibility in more complex evolutionary setups.

4.2.1 Hyperparameters

The GA explores the following hyperparameters:

- Minimum/maximum number of hidden layers
- Minimum/maximum neurons per hidden layer
- Population size
- Number of generations
- Crossover rate

- Mutation rates
- Tournament size

4.2.2 Individual Representation

The chromosome has fixed length based on the allowed range of hidden layers. Some genes may be inactive depending on the chosen topology. Gene semantics:

- Index 0: number of hidden layers (bounded by hyperparameters)
- Indices $i = 1$ to $i =$ *max hidden layers*: neurons in hidden layer i (bounded)
- Remaining genes: synaptic weights. Each layer occupies predetermined index ranges up to the maximum number of weights per layer, computed from the maximum neurons per layer.

4.2.3 Algorithm

We first used `deap.algorithms.eaSimple()`, the standard evolutionary loop: evaluate each individual, apply tournament selection, then crossover and mutation. The chromosome encodes both topology and weights.

We also tested a coevolutionary variant with two populations:

1. A topology population evolved while fixing the best weights.
2. A weight population evolved while fixing the best topology.

At each macro-generation, the topology population evolves for three micro-generations using the best weights; then the weight population evolves for three micro-generations using the best topology.

4.3 Genetic Neuroevolution with NEAT

We additionally used *neat-python* (MCINTYRE et al.,), which implements *NeuroEvolution of Augmenting Topologies* (NEAT) (STANLEY; MIIKKULAINEN, 2002). NEAT starts from a minimal topology (inputs and outputs) and adds neurons and connections by mutation.

4.3.1 Genome Representation

Genes represent nodes (neurons) or connections and include:

1. Innovation number (historical marker)
2. Synaptic weight
3. Enabled/disabled flag
4. If connection: the two endpoint nodes
5. If node: activation function

4.3.2 Mutation

Each generation, mutations may:

1. Perturb synaptic weights
2. Add a connection
3. Add a node
4. Change activation function (not in the output layer)
5. Enable or disable a connection

New nodes and connections receive new innovation numbers, which support consistent gene alignment during crossover and enable speciation.

4.3.3 Crossover

Parent genes are aligned by innovation number and classified as:

1. **Matching genes:** present in both parents.
2. **Disjoint genes:** present in only one parent, with innovation $\leq$ the other parent's maximum.
3. **Excess genes:** present in only one parent, with innovation $>$ the other parent's maximum.

Matching genes inherit weights from either parent at random; disjoint and excess genes are copied from the more fit parent (or from both in a tie).

4.3.4 Speciation

This algorithm also introduces the idea of **speciation** to protect new innovations, since a new node can, for instance, worsen the *fitness* in the short term, but become useful in the long run after its weight has evolved over several generations.

To this end, the notion of a compatibility distance, δ, between two different genomes is introduced, representing how different these two organisms are. If this distance is above a given threshold, δ_t, the two organisms are considered to belong to different species.

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \overline{W}$$

Where,

E: Number of excess genes

D: Number of disjoint genes

$\overline{W}$: Average difference of synaptic weights

N: Size of the larger genome

c_1, c_2, c_3: Tunable coefficients

To protect new innovations, the *fitness* of each organism is adjusted by dividing it by the size of its species. This penalizes large species and prevents a single species from dominating the entire population, while also giving new mutations time to develop.

Moreover, species are completely eliminated if they fail to improve within a given period (in this case, 15 generations), except for the three best species, which are protected by elitism to avoid a total wipeout of the population.

Finally, to allocate offspring, each species is first assigned a number of descendants proportional to the sum of the adjusted *fitness* of all its organisms. Then, the worst-performing organisms are removed and the remaining ones reproduce. The entire population of each species is replaced by its offspring, with the exception of the organisms protected by elitism (2 per species).

5 Results

Initial experiments used a fixed target position and compared three approaches:

1. DEAP — single population
2. DEAP — coevolution with separate topology and weight populations
3. NEAT

The input layer had 14 neurons (matching the observation space), and the output layer had 4 neurons with tanh activation, one per motor, each in $[-1, 1]$.

Further hyperparameter exploration is warranted and would require additional compute.

5.1 DEAP — Single Population

As shown in Fig. 1, the algorithm raised mean fitness over generations, with occasional improvements in the best individual. It did not reach the optimum: the best fitness was 49.3, close to the target but short of contact. Figure 2 shows premature convergence to a single hidden layer with 15 neurons.

This run used a population of 50. Networks could have from 1 to 10 hidden layers and 5 to 20 neurons per hidden layer. Mutation settings: 10 % chance to change the number of layers, 10 % chance to change the neuron count in a random layer, and Gaussian perturbations to synaptic weights.

5.2 DEAP — Coevolution

Figure 3 shows the coevolutionary setup (separate topology and weight populations), which achieved a best fitness of 58.2 and a mean of 34.9. Each population had 500 individuals and 100 macro-generations, each comprising 3 topology micro-generations followed by 3 weight micro-generations. Elitism preserved 10 % of the best parents.

5.3 NEAT

Because NEAT uses a compact internal network representation (without *keras*/*pytorch* overhead), runs were faster, enabling larger populations and broader sweeps. We used a population of 1500; activation choices included *sigmoid*, $ReLU$, and *tanh*.

In Fig. 4, fitness increases steadily, reaching a maximum of 98.49. The drone reached and stopped at the target; the remaining gap to the theoretical maximum (100) reflects travel time. Figure 5 zooms into the first 100 generations: rapid gains until roughly generation 20, followed by occasional improvements from mutation.

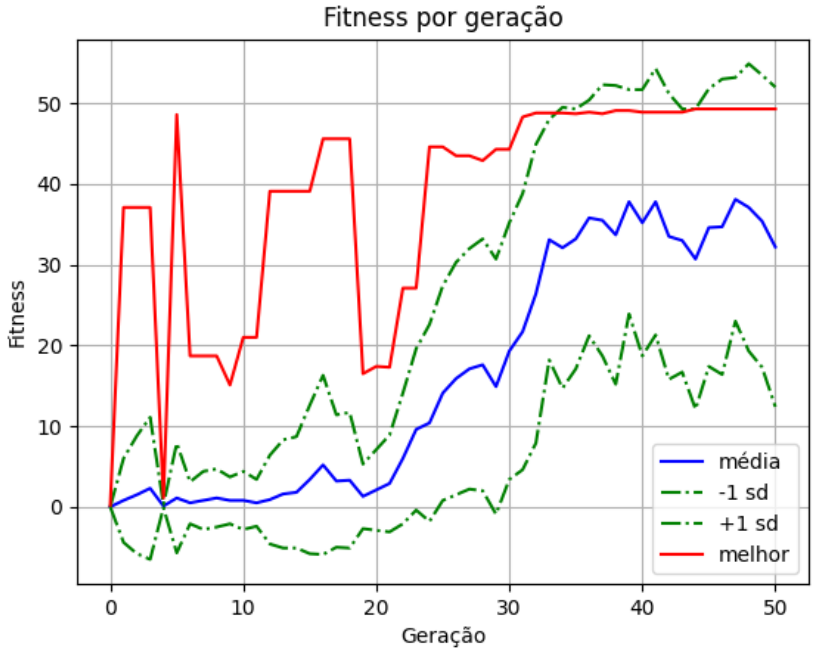
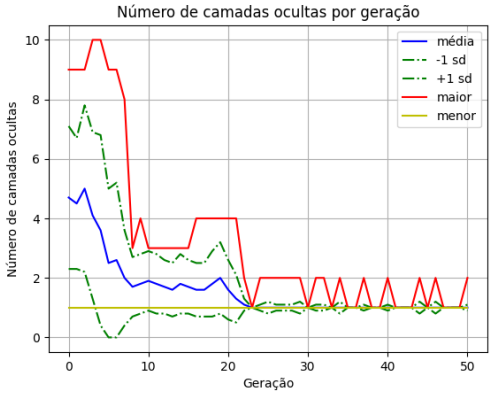
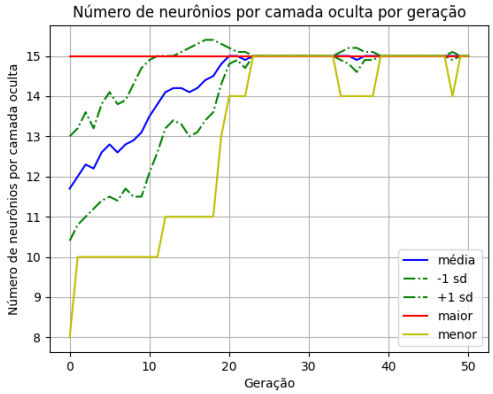


Figure 1: Performance of the single-population DEAP algorithm: fitness per generation.



(a) Hidden-layer count evolution



(b) Hidden-layer neuron count evolution

Figure 2: Topology evolution with the single-population DEAP algorithm.

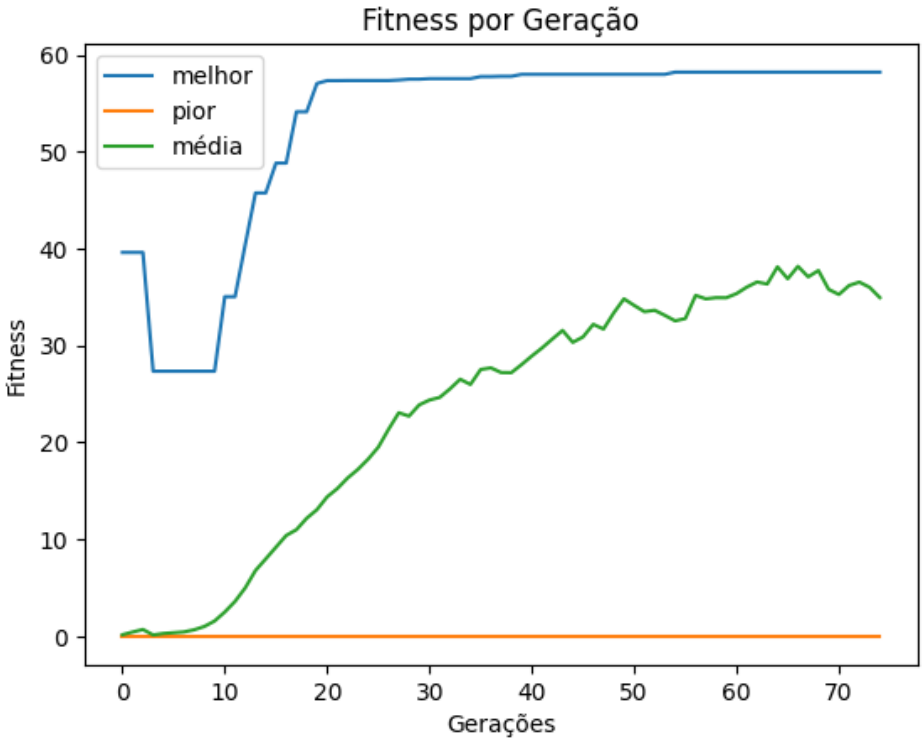


Figure 3: Coevolutionary algorithm: fitness per generation.

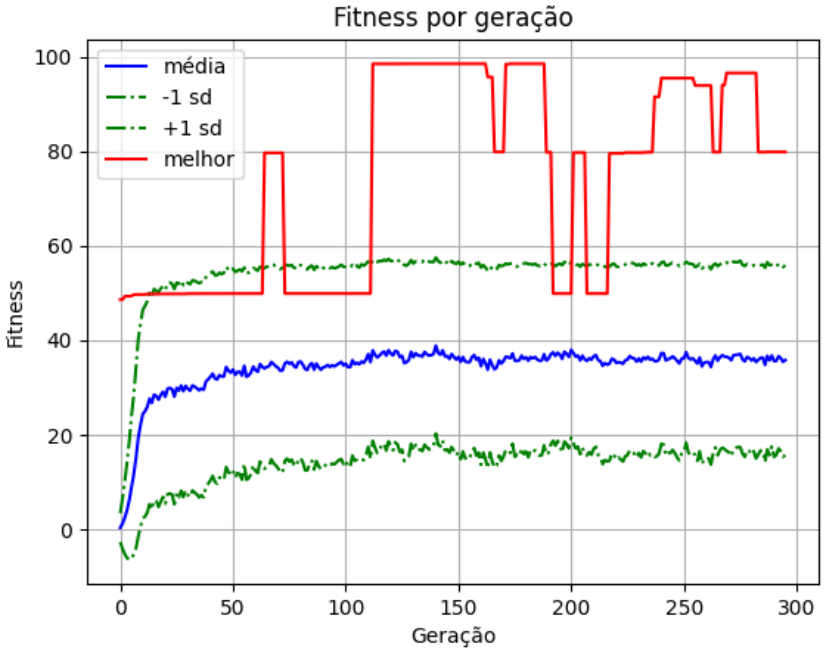


Figure 4: NEAT performance: fitness per generation.

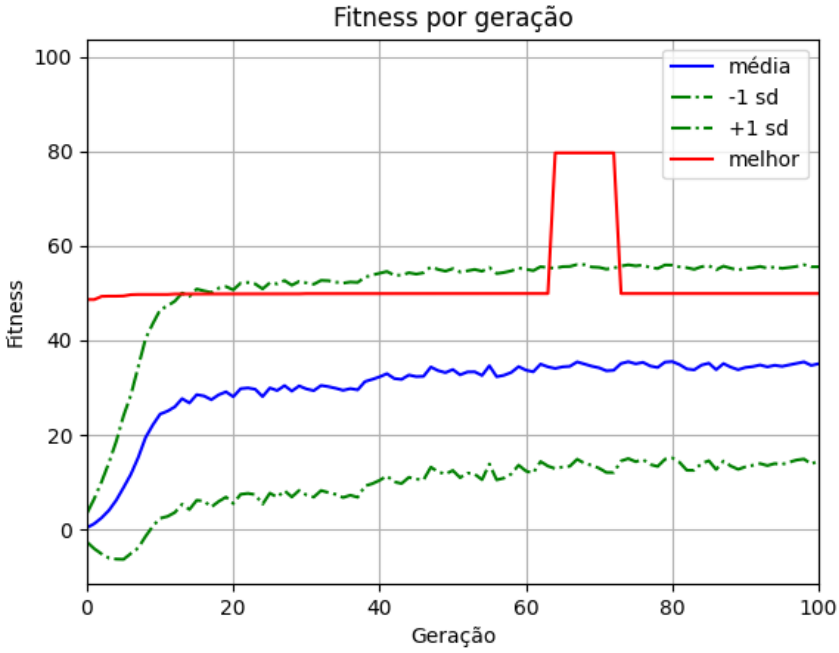
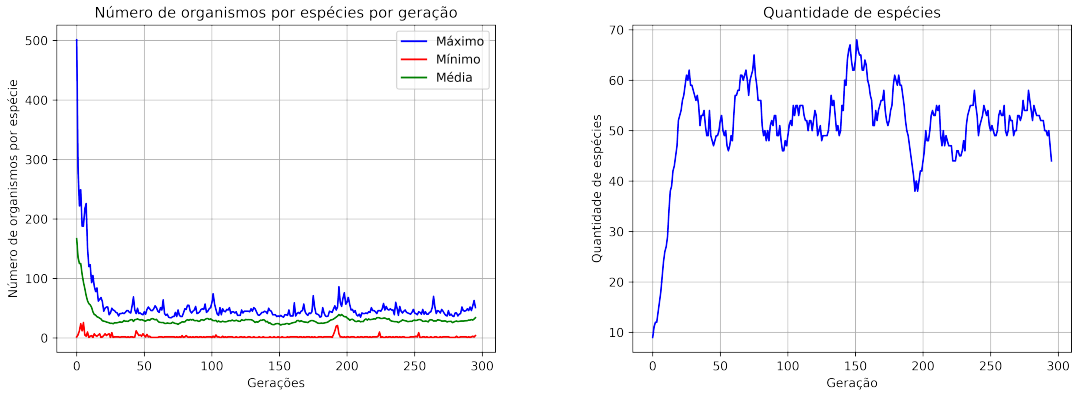


Figure 5: NEAT: fitness over the first 100 generations.

Figure 6 illustrates speciation dynamics. Panel 6a shows species sizes (min/mean/max per generation); Panel 6b shows the number of species per generation.



(a) Species size per generation

(b) Number of species per generation

Figure 6: Speciation dynamics (NEAT).

Finally, Fig. 7 depicts the highest-fitness network. Square nodes are inputs; round nodes indexed 0–3 are outputs; the remaining round nodes are hidden units. Green edges indicate positive weights and red edges negative weights; dashed edges are dis-

abled connections. The winning network included two hidden neurons with *ReLU* activation.

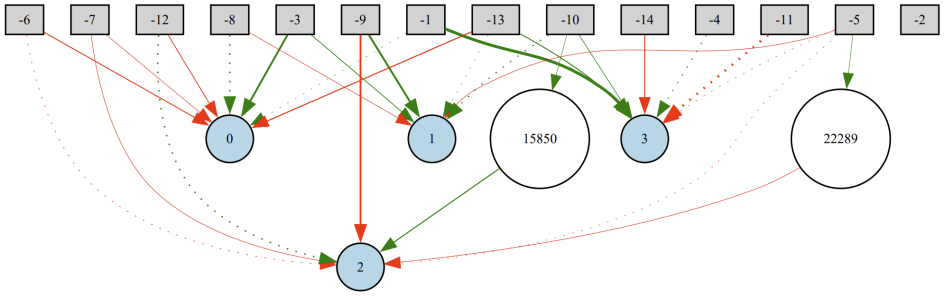


Figure 7: Architecture of the best-performing network evolved by NEAT.

References

BENARBIA, T.; KYAMAKYA, K. A literature review of drone-based package delivery logistics systems and their implementation feasibility. *Sustainability (Switzerland)*, v. 14, 2022. ISSN 20711050.

COUMANS, E.; BAI, Y. *PyBullet, a Python module for physics simulation for games, robotics and machine learning*. 2016–2019. ⟨http://pybullet.org⟩.

DAUD, S. M. S. M. et al. Applications of drone in disaster management: A scoping review. *Science and Justice*, v. 62, 2022. ISSN 18764452.

FÖRSTER, J. *System identification of the crazyflie 2.0 nano quadrocopter*. Dissertação (B.S. thesis) — ETH Zurich, 2015.

FORTIN, F.-A. et al. DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research*, v. 13, p. 2171–2175, jul 2012.

GAD, A. F. Pygad: An intuitive genetic algorithm python library. *CoRR*, abs/2106.06158, 2021. Disponível em: ⟨https://arxiv.org/abs/2106.06158⟩.

GEN, R. C. M. *Genetic Algorithms and Engineering Design*. 1. ed. [S.l.]: Wiley-Interscience, 1997. (Engineering Design and Automation). ISBN 0471127418; 9780471127413.

HAQUE, A.; CHOWDHURY, M. N.-U.-R. Exploring the benefits of reinforcement learning for autonomous drone navigation and control. *International Journal of Advanced Networking and Applications*, v. 15, 2023. ISSN 09750282.

KATOCH, S.; CHAUHAN, S. S.; KUMAR, V. A review on genetic algorithm: past, present, and future. *Multimedia Tools and Applications*, v. 80, 2021. ISSN 15737721.

MCINTYRE, A. et al. *neat-python*.

PANERATI, J. et al. Learning to fly—a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. [S.l.: s.n.], 2021.

REJEB, A. et al. Drones in agriculture: A review and bibliometric analysis. *Computers and Electronics in Agriculture*, v. 198, 2022. ISSN 01681699.

STANLEY, K. O.; MIIKKULAINEN, R. Evolving neural networks through augmenting topologies. *Evolutionary computation*, MIT Press, v. 10, n. 2, p. 99–127, 2002.